MDB100

Storage and Retrieval I

MongoDB Database and Security

Release: 20260220

Topics we cover

Creating Documents

Reading Documents

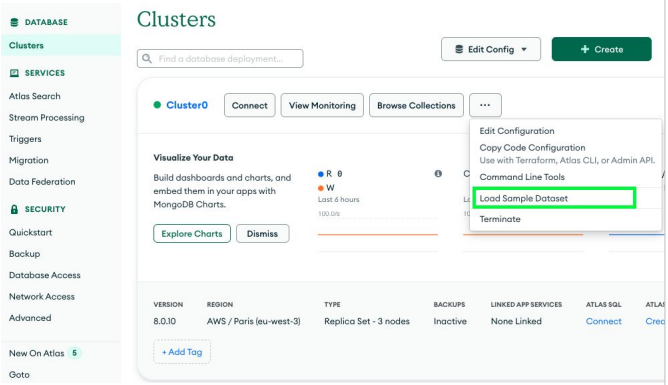
Cursors

Load Sample Data

In your Atlas cluster,
Click the three dots [...]
Select Load Sample Dataset

Click **Browse Collections** to view the databases and collections we loaded.

We will be using the **sample_training** database.



Follow the instructions to load sample data set in Atlas.
Click on the Collections Button to see a list of databases, out of which we would be using the **sample_training** database.

Validate Loaded Data

Connect to the MongoDB Atlas Cluster using mongosh

Verify that data is loaded

Database to use: **sample_training**

Collections to verify: **grades** and **inspections**

```
MongoDB> use sample_training
switched to db sample_training

MongoDB> db.grades.countDocuments({})
100000

MongoDB> db.inspections.countDocuments({})
80047

MongoDB>
```


Validate the loaded data by checking collection counts for **sample_training.grades** and **sample_training.inspections.**

countDocuments causes the query to return just the number of results found.

Basic Database CRUD Interactions

	Single Document	Multiple Documents
Create	`insertOne(doc)`	`insertMany([doc,doc,doc])`
Read	`findOne(query, projection)`	`find(query, projection)`
Update	`updateOne(query,change)`	`updateMany(query,change)`
Delete	`deleteOne(query)`	`deleteMany(query)`

MongoDB APIs allow us to perform Create Read Update and Delete operations options to perform single or multiple operations.

Anatomy of a MongoDB Document

A Document is an ordered set of field-and-value pairs

Every Document has an _id field

- The _id field's value must be unique within a collection

Values can be any BSON type (int, string, bool, array...)

Documents can contain sub-documents and arrays

Document representation in JavaScript

```
var mydoc = {
      _id:     ObjectId("50948bd2f98391"),

      name:     {
                  first: "Alan",
                  last: "Turing"
                },

      birth:    new Date('Jun 23, 1912'),
      death:    new Date('Jun 07, 1954'),

      contribs: [
                  "Turing machine",
                  "Turing test",
                  "Turingery"
                ],

      views:    NumberLong(1250000)
}
```


Ordered: fields are stored in the order they were created. (Some programming languages may not respect this order)
Set: each field name must be unique within a document
The _id ("underscore eye-dee") field acts like a primary key. Each document in a collection must have a unique _id value
Values can be any valid BSON type, including null.
Programming languages represent documents using native data structures, such as maps or dictionaries.

In Javascript, documents start and end with curly brackets ("braces"). Fields and values within a document are separated by a colon. Array elements and field-and-value pairs are separated by a comma. Arrays start and end with square brackets ("brackets").
Field names do not need to be in quotes UNLESS they contain special characters or spaces.

Creating Documents

insertOne() - Insert a Single Document

insertOne() adds a single document to a collection

_id value must be unique within a collection

Driver generates the _id value if not supplied

MongoDB responds with:

- Write Acknowledgment
- _id value of inserted document

```
test> use test
already on db test

test> db.movies.insertOne( {title: "Finding Nemo"} )
{
  acknowledged: true,
  insertedId: ObjectId('681cc5dc689817e718fe6911')
}

test> db.movies.insertOne( {_id: 0, title: "Back to the Future"}
 )
{ acknowledged: true, insertedId: 0 }

test> db.movies.insertOne( {_id: 0, title: "Back to the Future P
art II"} )
MongoServerError: E11000 duplicate key error collection: test.movies
index: _id_ dup key: { _id: 0 }
```

Practice:

```
test> db.movies.insertOne( {title: "Back to the Future Part III"} )
test> db.movies.insertOne( {name: "Edward Scissorhands"} )
test> db.movies.insertOne( {title: "Romeo + Juliet", year: "1996"} )
test> db.movies.insertOne( {name: "The Quiet American", released: 2002, directed_b
y: "Phillip Noyce" })
```


insertOne() adds a document to the collection on which it is called. It is the most basic way to add a new document to a collection.

The document to be inserted must be passed as a parameter to insertOne()

Because the _id field acts as a primary key, it must be unique within the collection

The write acknowledgement shows that the document has been written to disk. The client must wait for a response before it can continue. This behaviour is configurable - see the MDB300 course: https://learn.mongodb.com/courses/ilt-mdb300-mongodb-production-readiness for more information.

If the document does not contain an _id field, the driver will generate an ObjectID value for the _id field. The ObjectID contains an encoded timestamp, random value and incrementing counter. ObjectIDs are *likely unique*.

Try the example insertOne commands in your own Atlas cluster. Does MongoDB complain about the mix of field names? What problems might these cause in the future? Can you think of a better value to use for the _id field other than generated ObjectIds?

insertMany() - Insert Multiple Documents

insertMany() adds multiple documents to a collection

Accepts array of documents as a parameter

Bulk Write operation - reduces network round trips

MongoDB responds with

- Write Acknowledgment
- _id value of each inserted document
- Error report in case of problems

```
test> let movies = [ {title: "Gladiator"}, {title: "Ghostbusters
"}, {title: "Toy Story"} ]

test> db.movies.insertMany(movies)
{
  acknowledged: true,
  insertedIds: {
      '0': ObjectId('681dd457689817e718fe8086'),
      '1': ObjectId('681dd457689817e718fe8087'),
      '2': ObjectId('681dd457689817e718fe8088')
  }
}
```

Practice:

```
// use the up-arrow in mongosh to access your command history
test> let movies = [ {_id: 0, title: "Ghost"}, {title: "Toy Story 2"} ]
test> db.movies.insertMany(movies)
```


insertMany() can add multiple new documents to a single collection.
This avoids the need for a network round trip per document, which is really slow.
If all inserts were successful, MongoDB returns a document showing all inserted _ids
Limit of 48MB in a single call to the server. Driver will automatically split larger batches into multiple 48MB batch inserts
There is a way to bundle Insert, Update and Delete operations into a single network call too called BulkWrite.
Use dedicated tools like mongoimport or mongorestore to import raw data or multiple collections

Try the example insertMany commands in your own Atlas cluster. Assume there is already a document with _id:0 in the movies collection. Is “Toy Story 2” added to the movies collection after the second insertMany command?

Ordered Inserts and Unordered Inserts

insertMany() inserts documents in order by default

Add parameter `{ordered:false}` for an unordered insert

Ordered Inserts (default) **stop** on first error.

Unordered Inserts:
- report errors
- continue inserting
- can execute in parallel on sharded clusters

```
test> let friends = [
        {_id: "joe" },
        {_id: "bob" },
        {_id: "joe" },
        {_id: "jen" }
]

test> db.collection1.insertMany(friends)
{ errmsg : "E11000 duplicate key error ...",
nInserted : 2 }

test> db.collection2.insertMany(friends, {ordered:false})
{ errmsg : "E11000 duplicate key error ...",
nInserted : 3 }

test> db.collection1.find()
{ _id : "joe" }
{ _id : "bob" }

test> db.collection2.find()
{ _id : "joe" }
{ _id : "bob" }
{ _id : "jen" }
```


Try the commands in your own Atlas cluster to see the full error messages. Note that you get a much more detailed response if an error occurs while inserting in bulk. Look at the "BulkWriteResult" object. What does it tell you about the bulk operations that MongoDB supports?

Reading Documents

findOne() - Retrieve One Document

findOne() retrieves a single document

Accepts a **query** document
- Query can be empty
- Query can contain multiple fields

Returns the **first** match, or `null`
- Queries are **type**-sensitive
- Queries are **case**-sensitive

```
test> use sample_training
switched to db sample_training

sample_training> db.zips.findOne( {} )
{
  _id: ObjectId('5c8eccc1caa187d17ca6ed1d'),
  city: 'BREMEN',
  zip: '35033',
  loc: { y: 33.973664, x: 87.004281 },
  pop: 3448,
  state: 'AL'
}

sample_training> db.zips.findOne( {city: "BREMEN"} )
{
  _id: ObjectId('5c8eccc1caa187d17ca6ed1d'),
  city: 'BREMEN',
  zip: '35033',
  loc: { y: 33.973664, x: 87.004281 },
  pop: 3448,
  state: 'AL'
}

Practice:
sample_training> db.zips.findOne( {state: "AL", pop: 3448 } )
sample_training> db.zips.findOne( {city: "Bremen"} )sample_training> db
.zips.findOne( {zip: 35033} )
sample_training> db.zips.findOne( {STATE: "AL"} )
```


findOne() returns a single document which matches the query parameter.

The query parameter is an optional document containing any number of selection criteria.

MongoDB returns the **first** document which matches the query.
This match is **non-deterministic** unless the query uses an index, so your Atlas environment may show a different document to the one shown on the slide.
If there are no matches, the command returns 'null'.
NB: Not all language drivers implement findOne.

MongoDB will only match documents where the type is the same in the query and the document, and where the field names are exactly the same in the query and the document. String matches for values are case sensitive as well.

Try the example queries in your own Atlas environment with the sample data loaded. Try to predict the outcome of the queries before you run them.

Regular Expression in Queries

- Regex can be used to find string values without needing exact matching related to case sensitivity
- $regex operator is optional in syntax and can be omitted to get the same result
- Case-insensitive regex queries cannot use an index
- Use Atlas Search for full-text searches in data stored in MongoDB Atlas

```
MongoDB> db.zips.findOne({city: "Bremen"}) // No match ✗

MongoDB> db.zips.findOne({city: {$regex: /Bremen/i}})
//Returns a match

MongoDB> db.zips.findOne({city: /Bremen/i})
//Returns a match
```


Regular expressions are powerful but can also be costly. They require careful use of MongoDB text indexes for acceptable performance.

If you are using MongoDB Atlas, consider using Atlas Search for full-text searches. Atlas Search also has its own $regex operator. More information is available in the IL T DS110: Atlas Search and Vector Search course.

Projection: choosing Fields to Return

Find operations can include a **projection** document

Projections define fields to include OR exclude

- _id field is a special case

Projected Queries return a **subset** of each document

- Reduces **network traffic**
- Reduces client memory requirements
- MongoDB must still read the **entire document**

```
sample_training> db.zips.findOne( {city: "BREMEN"}, {city:1} )
{
  _id: ObjectId('5c8eccc1caa187d17ca6ed1d'),
  city: 'BREMEN'
}

sample_training> db.zips.findOne( {city: "BREMEN"}, {city:0} )
{
  _id: ObjectId('5c8eccc1caa187d17ca6ed1d'),
  zip: '35033',
  loc: { y: 33.973664, x: 87.004281 },
  pop: 3448,
  state: 'AL'
}

sample_training> db.zips.findOne( {city: "BREMEN"}, {city:1, _id
:0} )
{
  city: 'BREMEN'
}
```

Practice:

```
sample_training> db.zips.findOne( {city: "BREMEN"}, {city:1, state:1} )
sample_training> db.zips.findOne( {city: "BREMEN"}, {city:1, pop:1, _id:0} )
sample_training> db.zips.findOne( {city: "BREMEN"}, {city:1, zip:1, loc:0} )
sample_training> db.zips.findOne( {city: "BREMEN"}, {loc:1} )
```


MongoDB Documents can be up to 16Mb in size, but we may not always need every field-and-value pair returned.

We can specify which fields we want MongoDB to return in a projection document.

We can select the fields to return by providing an object with those fields and a value of 1 for each.

We can instead choose what fields NOT to return by providing an object with fields set to 0.

We cannot mix and match 0 and 1 - as what should it do with any other fields?

There is an exception where we can use **_id: 0** to exclude **_id** from the projection and include only the fields that are required { _id:0, name : 1 }

Projections are especially useful for excluding large sub-documents from responses. However, MongoDB must still read and cache the entire document.

Try the example queries in your own Atlas environment with the sample data loaded. Try to predict the outcome of the queries before you run them.

Retrieve Multiple Documents

find() returns a cursor object

Iterate through the cursor to retrieve matches

mongosh iterates through the cursor automatically

We can exhaust the cursor and fetch all matches using `.toArray()`

```
sample_training> db.zips.find( {state: "NY"} )
[
  {
      _id: ObjectId('5c8eccc1caa187d17ca72f89'),
      city: 'FISHERS ISLAND',
      zip: '06390',
      loc: { y: 41.263934, x: 72.017834 },
      pop: 329,
      state: 'NY'
  },
...
]
Type "it" for more

sample_training> db.zips.find( {state: "NY"}, {loc:0, _id:0} ).t
oArray()
[
  { city: 'HARRISON', zip: '10528', pop: 11601, state: 'NY' },
  { city: 'MAHOPAC', zip: '10541', pop: 23687, state: 'NY' },
  { city: 'KATONAH', zip: '10536', pop: 12263, state: 'NY' },
  { city: 'IRVINGTON', zip: '10533', pop: 7498, state: 'NY' },
  ... 1496 more items
]
```


find() returns all documents which match the query

find() takes an optional query document and an optional projection document as parameters, similar to findOne()

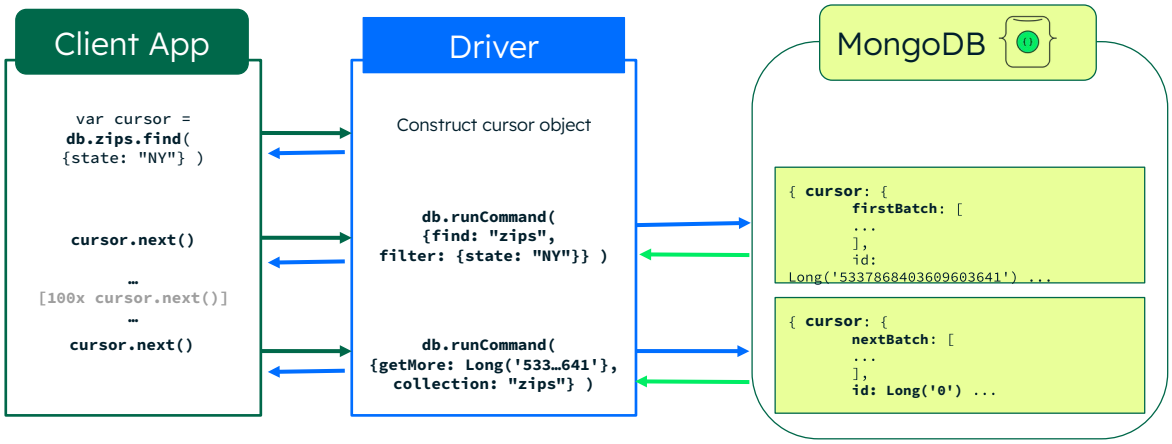
Mongosh automatically iterates through the cursor object and shows the results in batches.

Try and find out how many documents mongosh returns per batch.

Why might using .toArray() to exhaust the cursor and return all matches **not** be a good idea?

Cursors

Returning Results



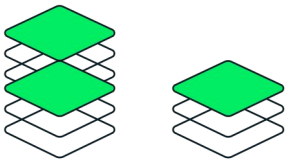
- mongosh's find() is a helper method for the MongoDB "find" command.
- The database responds to find() with the first 101 documents or 16MB of results (whichever comes first). These are typically cached by the driver.
- For more results, the driver sends the cursor ID with a `getMore` command to retrieve the next 16MB.
- `getMore` continues until a cursor ID of 0 signifies completion.
- Drivers typically handle `getMore` automatically when iterating the cursor.
- Some drivers delay sending the "find" command until cursor iteration begins.

Cursors work in batches

Cursors fetch results from the server in batches.

The default batch size in the shell is 101 documents during the initial call to find() with a limit of 16MB.

If we fetch more than the first 100 document from a cursor it fetches in 16MB batches in the shell or up to 48MB in some drivers.



Rather than make a call to the server every time we get the next document from a cursor, the server fetches the result in batches and stores them at the client or shell end until we want them.

Fetching documents one by one would be slow.

Fetching all documents at once would use too much client RAM.

We can change the batch size on the cursor if we need to but it's still limited to 16M.

Fetching additional data from a cursor uses a function called getmore() behind the scenes, it fetches 16MB at a time.

Using Cursors

Here, we store the result of find to a variable.

We then manually iterate over the cursor.

The query is not actually run until we fetch results from the cursor.

```
MongoDB> for(let x=0;x<200;x++) {
       db.taxis.insertOne({ plate: x })
}
MongoDB> let mycursor = db.taxis.find({})

MongoDB> while (mycursor.hasNext()) {
       let doc = mycursor.next();
       printjson(doc)
}
{ _id : ObjectId("609b9aaccf0c3aa225ce9117"), plate : 0 }
{ _id : ObjectId("609b9aaccf0c3aa225ce9118"), plate : 1 }
   ...
{ _id : ObjectId("609b9aaccf0c3aa225ce91dd"), plate : 199 }

MongoDB> let mycursor = db.taxis.find({}) // No Output

MongoDB> mycursor.forEach( doc => { printjson(doc) })

//This does nothing - does not even contact the server!
MongoDB> for(let x=0;x<100;x++) {let c = db.taxis.find({})}
```


`mycursor` is a cursor object, it knows the database, collection and query we want to run.

Until we do something with it it has not run the query - it has not even contacted the server.

It has methods - importantly , in mongosh `hasNext()` and `next()` to check for more values and fetch them.

We can iterate over a cursor in various ways depending on our programming language.

If we don't fetch information from a cursor - it never executes the find - this might not be expected when doing simple performance tests like the one below.

To pull the results from a cursor in a shell for testing speed we can use `db.collection.find(query).itcount()` - see https://www.mongodb.com/docs/manual/reference/method/cursor.itcount/

Cursor modifiers

Cursors can include additional instructions like **limit**, **skip**, etc.

Skip and limit return us cursors.

```
MongoDB> db.taxis.find({}).limit(5)
{ _id : ObjectId("609b9aaccf0c3aa225ce9116"), plate : 0 }
{ _id : ObjectId("609b9aaccf0c3aa225ce9117"), plate : 1 }
{ _id : ObjectId("609b9aaccf0c3aa225ce9118"), plate : 2 }
{ _id : ObjectId("609b9aaccf0c3aa225ce9119"), plate : 3 }
{ _id : ObjectId("609b9aaccf0c3aa225ce911a"), plate : 4 }

MongoDB> db.taxis.find({}).skip(2)
{ _id : ObjectId("609b9aaccf0c3aa225ce9118"), plate : 2 }
...
{ _id : ObjectId("609b9aaccf0c3aa225ce912b"), plate : 21 }
Type "it" for more

MongoDB> db.taxis.find({}).skip(8).limit(2)
{ _id : ObjectId("609b9aaccf0c3aa225ce911e"), plate : 8 }
{ _id : ObjectId("609b9aaccf0c3aa225ce911f"), plate : 9 }
```


We can add a limit instruction to the cursor to stop the query when it finds enough results.

We can add a skip instruction to the cursor to tell it to ignore the first N results.

Skip is always performed before the limit when computing the answer.

This can be used for simple paging of results - although it's not the optimal way of doing so.

Skip has a cost on the server - skipping a large number of documents is not advisable.

Sorting Results

Use **sort()** cursor modifier to retrieve results in a specific order

Specify an object listing fields in the order to sort and sort direction

```
MongoDB> let rnd = (x)=>Math.floor(Math.random()*x)

MongoDB> for(let x=0;x<100;x++) { db.scores.insertOne({ride:rnd(
40),swim:rnd(40),run:rnd(40)})}

//Unsorted
MongoDB> db.scores.find({},{_id:0})
{ ride : 5, swim : 11, run : 11 }
{ ride : 0, swim : 17, run : 12 }
{ ride : 17, swim : 2, run : 2 }

//Sorted by ride increasing
MongoDB> db.scores.find({},{_id:0}).sort({ride: 1})
{ ride : 0, swim : 38, run : 10 }
{ ride : 1, swim : 37, run : 37 }
{ ride : 1, swim : 30, run : 20 }

//Sorted by swim increasing then ride decreasing
MongoDB> db.scores.find({},{_id:0}).sort({swim: 1, ride: -1})
{ ride : 31, swim : 0, run : 14 }
{ ride : 11, swim : 0, run : 14 }
{ ride : 30, swim : 1, run : 34 }
{ ride : 21, swim : 1, run : 3 }
```


With Skip and Limit sorting is important, because we cannot assume anything about the order of unsorted results.

Sorting results without an index is very inefficient - we cover this when talking about indexes later.

Exercise

Add four documents to a collection called `diaries` using the commands shown here.

Write a find() operation to output only diary entries from **dug**.

Modify it to output the line below using **skip**, **limit** and a **projection**.

```
[{name: 'dug', txt: 'saw a squirrel'}]
```

```
MongoDB> db.diaries.drop()

MongoDB> db.diaries.insertMany([
        {
                name: "dug", day: ISODate("2014-11-04"),
                txt: "went for a walk"
        },
        {
                name: "dug", day: ISODate("2014-11-06"),
                txt: "saw a squirrel"
        },
        {
                name: "ray", day: ISODate("2014-11-06"),
                txt: "met dug in the park"
        },
        {
                name: "dug", day: ISODate("2014-11-09"),
                txt: "got a treat"
        }
])
```


Answers at the end

Bonus Exercise: Combined Query

Use the **sample_training.companies** collection to find the largest company (by number of employees) that has fewer than 200 employees*.

Write a query that prints out only the company name and number of employees.

1. Use find() with a query and a projection
2. Apply some of the cursor operators: sort(), skip() and limit()

* Use the $lt operator (less than) to filter the number of employees

Answers at the end
Note: the Range operators, like the "less than" operator ($lt), are reviewed in Storage and Retrieval Part II

Quiz Time!

#1. When does a find() query get executed on the MongoDB server?

A	When a cursor is iterated	B	When you call the find() function	C	When the driver connects to the database
D	Every time we add a projection	E	Every time an index is created		

Answer in the next slide.

#1. When does a find() query get executed on the MongoDB server?

A	When a cursor is iterated
B	When you call the find() function
C	When the driver connects to the database
D	Every time we add a projection
E	Every time an index is created

find() returns a cursor object rather than a document/s. The shell starts retrieving the first 20 results but by default find() on its own does not retrieve documents.
Calling find() does not return any values until you start retrieve data with the cursor.
The find() query does not have relationship with a connection pool or the driver connection.
The creation of a cursor, adding a projection, or creating an index do not execute the find query.

#2. Why is insertMany() faster than multiple insertOne() operations?

A	Needs fewer writes to disk.
B	Reduces the network time.
C	Performs the writes as a single transaction.
D	Replicates to other servers faster.
E	Allows parallel processing of inserts in sharded clusters.

Answer in the next slide.

#2. Why is insertMany() faster than multiple insertOne() operations?

A Needs fewer writes to disk.

B Reduces the network time.

C Performs the writes as a single transaction.

D Replicates to other servers faster.

E Allows parallel processing of inserts in sharded clusters.

InsertMany can load documents in parallel if we had a sharded cluster with the option of (ordered:false). Since it is a bulk operation, it can batch process upto 48MB or 100,000 documents into a single round trip call to the db server, thus reducing the network time needed. It is not a single atomic operation however, as each individual document is inserted atomically.

#3. What metric is **reduced** when using a projection?

A	Amount of data read by MongoDB	B	Amount of data transmitted by MongoDB	C	Amount of memory used by MongoDB
D	Amount of CPU used by MongoDB	E	Number of indexes used by the query		

Answer in the next slide.

#3. What metric is **reduced** when using a projection?

A	Amount of data read by MongoDB
B	Amount of data transmitted by MongoDB
C	Amount of memory used by MongoDB
D	Amount of CPU used by MongoDB
E	Number of indexes used by the query

A projection only reduces the amount of data which is transmitted over the network from MongoDB to the client application/driver. The database will still read and cache the entire document as normal. The only exception to this is when using an “index-covered query”. This is covered in the Indexes module in MDB200.

Recap

Using Bulk writes vs. Single Writes has better network performance

find() returns us a cursor object which the shell then pulls from

Exercise Answers

Answer -Exercise: find, skip and limit

Write a find() to output only diary entries from "dug":

```
MongoDB> db.diaries.find({name:"dug"})

{"_id" : ObjectId("609ba812cf0c3aa225ce91de"), "name" : "dug", "day" : ISODate("2014-11-04T00:00:00Z"), "txt" : "went for a walk" }

{"_id" : ObjectId("609ba812cf0c3aa225ce91df"), "name" : "dug", "day" : ISODate("2014-11-06T00:00:00Z"), "txt" : "saw a squirrel" }

{"_id" : ObjectId("609ba812cf0c3aa225ce91e1"), "name" : "dug", "day" : ISODate("2014-11-09T00:00:00Z"), "txt" : "got a treat" }
```

Modify it to output the line below using skip, limit and a projection:

```
MongoDB> db.diaries.find({name:"dug"},{_id:0,day:0}).skip(1).limit(1)

{ name: "dug", txt: "saw a squirrel" }
```

Answer -Exercise: Combined Query

What company in the companies collection that has fewer than 200 employees has the most employees?

```
MongoDB> db.companies.find(
    {number_of_employees:{$lt:200}},
    {number_of_employees:1, name:1, _id: 0}
).sort({number_of_employees:-1}).limit(1)
```

```
{name: 'uShip',number_of_employees: 190}
```